

REST 練習 6：POST 建立資源

Lecture

前五題都是 **GET**——只查詢、不改變伺服器狀態。這題第一次示範會改變資料的動詞：**POST**。

POST 的標準語意是「在這個集合底下建立一筆新資源」，跟 **PUT**（整筆覆蓋更新一個已知身分的資源）不同：**POST** 的呼叫端通常不知道新資源最終的身分（URI）會是什麼，是伺服器決定的；**PUT** 的呼叫端已經知道要更新的是哪一筆（URI 裡已經包含識別碼）。也因為這樣，**POST** 天生**不是幂等的**——同樣的請求送兩次，理論上會建立出兩筆不同的資源（這題用主鍵重複偵測、回 **409** 來補這個洞，見思考題 3 的延伸討論）。

201 Created 搭配 **Location** header 是 REST 對「成功建立資源」的標準回應方式：狀態碼告訴呼叫端「東西建好了」，**Location** header 告訴呼叫端「東西在這裡，之後要查詢/更新/刪除就用這個網址」——這樣呼叫端不用自己再組一次 URL 規則，伺服器怎麼決定新資源的網址是伺服器的實作細節，呼叫端只要照著 **Location** 走就好。

這題也把 rs04 的 **404**、加上這題新出現的 **400**（呼叫端送來的資料本身不合法）、**409**（請求沒錯但跟目前系統狀態衝突）湊成一組 REST 最常用的 4xx 家族，之後設計任何 API 時，分清楚「這個錯誤情境該回哪一個」是基本功。

至於 **CSRF 保護**，是另一個 **POST/PUT/DELETE**（會改動資料的動詞）才需要考慮的機制——因為 **GET** 語意上不該有副作用，瀏覽器允許跨站請求夾帶 Cookie 呼叫 GET 相對安全；但換成會寫入資料的動詞，就要防範「使用者不知情的情況下被誘導發出請求」，這正是 CSRF Token 機制存在的理由（詳細說明見下方「為什麼要 CSRF 保護」）。

學習目標

- 理解 **IF_REST_RESOURCE~POST** 的 **IO_ENTITY** 參數就是 Request Body：框架的 **CL_REST_RESOURCE~DO_HANDLE** 會自動呼叫 **POST(mo_request->get_entity())**，資源類別覆寫 **POST** 時直接拿 **IO_ENTITY** 用，不用自己再去 **mo_request** 掏一次
- 用 **IO_ENTITY->GET_STRING_DATA()** 讀出 Request Body 的原始 JSON 字串，再用 **/UI2/CL_JSON=>DESERIALIZE()** 反序列化成 ABAP 結構——跟 rs03~rs05 一路用的 **SERIALIZE**（ABAP → JSON）方向相反
- 學會 **201 Created** 的正確語意：不是隨便回一個 2xx 就好，**成功建立資源要在回應標頭加 Location**，告訴呼叫端「你剛建立的資源，之後可以用這個網址查詢/操作」——這題的 **Location** 值直接沿用 rs04 教過的單筆查詢路徑格式 **/flights/{carrid}/{connid}/{fldate}**
- 認識 REST 的 **CSRF (Cross-Site Request Forgery)** 保護機制：**CL_REST_HTTP_HANDLER** 內建 **HANDLE_CSRF_TOKEN**，對 GET/HEAD/OPTIONS 以外的動詞（POST/PUT/DELETE）會要求先用 GET 帶 **X-CSRF-Token: fetch** 換一個 token，再把這個 token 放進後續請求的 header 才會放行；子類別可以覆寫這個方法、留空、不呼叫 **SUPER->** 來跳過檢查，但**這是有代價的選擇，要清楚知道什麼情境才適合關閉**（見下方「為什麼要 CSRF 保護、這題為什麼關掉它」）
- 分辨這題新出現的 **400 Bad Request** 跟 rs04/rs05 教過的 **404 Not Found** 差在哪裡：**404** 是「呼叫端要讀取一個不存在的資源」，**400** 是「呼叫端送來的資料本身有問題（缺欄位、引用了不存在的外鍵）」——同樣是「carrid 不存在」，rs04/05 的 GET 情境回 404，這題 POST 建立資源時卻回 400，因為情境不同：GET 是在找一個東西，POST 是在檢查你給的輸入合不合法

為什麼要 CSRF 保護、這題為什麼關掉它

CSRF 保護要防的是：使用者已經登入某個 SAP 系統（瀏覽器帶著有效的 session cookie），這時如果不小心開了一個惡意網頁，那個網頁可以偷偷用使用者的瀏覽器對 SAP 系統發出 POST/PUT/DELETE 請求（瀏覽器會自動帶上 session cookie，看起來就像使用者本人在操作）——X-CSRF-Token 這個機制要求呼叫端「先證明你有能力讀到系統的回應」（GET 一個 token 回來）才能送出變更請求，惡意網頁做不到這一步（跨網域的 GET 回應瀏覽器不會讓它讀到），藉此擋下偽造請求。

這是只有「瀏覽器 + Cookie-based session」這種情境才需要的防護：這題用 SPROX_HTTP_REQUEST（SAP GUI 內的標準測試程式）呼叫，它不是瀏覽器、也不會自動處理「先 GET 拿 token、再帶進 POST header」這個兩階段流程，如果不關閉 CSRF 檢查，ZCL_RS06_FLIGHTS~POST 每次都會先被 HANDLE_CSRF_TOKEN 擋下、回 403，示範不了 POST 本身的邏輯。所以這題的 ZCL_RS06_APP 覆寫 HANDLE_CSRF_TOKEN 成空實作（不呼叫 SUPER->）跳過檢查。

這個決定只適合本課程的測試情境，正式的、給瀏覽器呼叫的 API 不應該這樣做：如果一支 API 明確只給 Server-to-Server 呼叫（例如用 Basic Auth 或 OAuth Client Credentials，完全不依賴瀏覽器 session cookie），關閉 CSRF 檢查是合理的（因為攻擊手法的前提「瀏覽器自動帶 cookie」根本不成立）；但如果 API 會被瀏覽器前端呼叫（登入後的 session 用 cookie 維持），關閉 CSRF 保護就是把系統暴露在跨站偽造請求的風險下。

事前準備

ADT 端物件已由課程準備好（\$TMP）：

- ZCL_RS06_APP——Application Class，router 掛一個資源 /flights；覆寫 HANDLE_CSRF_TOKEN 跳過 CSRF 檢查
- ZCL_RS06_FLIGHTS——POST 建立一筆 SFLIGHT：讀 Request Body → JSON 反序列化 → 驗證必填欄位與 carrid 有效性 → 寫入 → 201 Created + Location header

題目需求（對照已建好的答案物件）

1. ZCL_RS06_APP 的 GET_ROOT_HANDLER：跟前幾題一樣，router 掛一條路徑

```
``abap DATA(lo_router) = NEW cl_rest_router( ). lo_router->attach( iv_template =
'/flights' iv_handler_class = 'ZCL_RS06_FLIGHTS' ). ro_root_handler = lo_router. ``
```

1. ZCL_RS06_APP 覆寫 HANDLE_CSRF_TOKEN（PROTECTED SECTION 宣告 METHODS handle_csrf_token REDEFINITION.）：

```
``abap METHOD handle_csrf_token.
```

- 刻意留空、不呼叫 SUPER->，跳過 CSRF Token 檢查

```
ENDMETHOD. ``
```

1. ZCL_RS06_FLIGHTS 的 POST：

```
``abap TYPES: BEGIN OF ty_flight, carrid TYPE s_carr_id, connid TYPE s_conn_id, fldate TYPE s_date, price TYPE
s_price, currency TYPE s_currcode, END OF ty_flight.
```

```
DATA(lv_body) = io_entity->get_string_data( ).
```

```
DATA ls_flight TYPE ty_flight. /ui2/cl_json=>deserialize( EXPORTING json = lv_body pretty_name =  
/ui2/cl_json=>pretty_mode-camel_case CHANGING data = ls_flight ).
```

```
IF ls_flight-carrid IS INITIAL OR ls_flight-connid IS INITIAL OR ls_flight-fldate IS INITIAL. RAISE EXCEPTION TYPE  
cx_rest_resource_exception EXPORTING status_code = cl_rest_status_code=>gc_client_error_bad_request  
request_method = if_rest_message=>gc_method_post. ENDIF.
```

```
SELECT SINGLE carrid FROM scarr WHERE carrid = @ls_flight-carrid INTO @DATA(lv_carrid_check). IF sy-subrc  
<> 0. RAISE EXCEPTION TYPE cx_rest_resource_exception EXPORTING status_code =  
cl_rest_status_code=>gc_client_error_bad_request request_method = if_rest_message=>gc_method_post.  
ENDIF.
```

```
DATA ls_sflight TYPE sflight. ls_sflight = CORRESPONDING #( ls_flight ).
```

```
INSERT sflight FROM ls_sflight. IF sy-subrc <> 0. RAISE EXCEPTION TYPE cx_rest_resource_exception  
EXPORTING status_code = cl_rest_status_code=>gc_client_error_conflict request_method =  
if_rest_message=>gc_method_post. ENDIF.
```

```
DATA(lv_fldate_str) = CONV string( ls_flight-fldate ). DATA(lv_location) = '/flights/' && ls_flight-carrid && '/'  
&& ls_flight-connid && '/' && lv_fldate_str.
```

```
mo_response->set_header_field( iv_name = 'Location' iv_value = lv_location ). mo_response->set_status(  
cl_rest_status_code=>gc_success_created ).
```

```
DATA(lv_json) = /ui2/cl_json=>serialize( data = ls_flight pretty_name = /ui2/cl_json=>pretty_mode-camel_case  
).
```

```
DATA(lo_resp_entity) = mo_response->create_entity( ). lo_resp_entity->set_string_data( lv_json ).  
lo_resp_entity->set_content_type( if_rest_media_type=>gc_appl_json ). ``
```

幾個實作細節：

- **carrid/connid/fldate** 三個都必填（缺任一個都回 400），因為它們合起來是 **SFLIGHT** 的主鍵，沒有它們無法定位這筆新資源；**price/currency** 不強制檢查，缺了就是空值/0，不影響能不能建立這筆記錄
- **carrid** 驗證存在性的邏輯跟 **rs04/rs05** 完全一樣（**SELECT SINGLE ... FROM scarr**），但這裡查不到回的是 **400**，**rs04/rs05** 的 **GET** 情境查不到回的是 **404**——同一段驗證邏輯，因為所在的 HTTP 動詞語意不同，該回的狀態碼也不同，這是這題特別想讓你對照的地方
- **INSERT sflight FROM ls_sflight** 用的是完整 **SFLIGHT** 型別的工作區，不是直接拿 5 欄位的 **ls_flight** 硬塞：**CORRESPONDING #(ls_flight)** 把 5 個有填的欄位轉進一個完整 **SFLIGHT** 結構，其餘欄位（**PLANETYPE**、**SEATSMAX** 等）留空/0——如果直接寫 **INSERT sflight FROM CORRESPONDING sflight(ls_flight)**（把目標型別名稱直接寫在 **CORRESPONDING** 後面、內嵌在 **INSERT ... FROM** 子句裡）語法檢查會報 "**SFLIGHT**" is not allowed here，**CORRESPONDING** 的目標型別要嘛用 #（從賦值左邊推斷）、要嘛先賦值給一個宣告好型別的變數，不能直接內嵌一個顯式型別名稱在這種語境下使用（2026-07-12 出這題時實測踩到的坑）
- **INSERT** 失敗（**sy-subrc <> 0**）代表主鍵已存在（重複建立），回 **409 Conflict**——這是 REST 的標準用法：對「這個資源已經存在，不能用 POST 重複建立」這種情境，**409** 比 **400** 更精確地表達「你的請求本身沒錯，只是跟目前系統狀態衝突」

- **Location header** 的日期用 `CONV string( ls_flight-fldate )`，不是字串樣板 `|{ ls_flight-fldate }|` 直接內插：ABAP 字串樣板對 **DATS** 型別的值預設會套用使用者的日期顯示格式（可能變成帶點的 `01.15.2026` 之類），`CONV string( ... )` 才會保留 **DATS** 內部儲存的原始 8 碼數字（`20260115`），這樣組出來的 **Location** 網址才能直接拿去給 **rs04** 的路徑參數用（**rs04** 的 `CONV dats( ... )` 期待的正是這種不帶分隔符的 8 碼格式）

SICF 掛載步驟（比照 rs01~rs05）

沿用既有的 `/sap/bc/zrest_training` 分類節點，這題只要在其底下再加一個子節點：

1. SICF 交易碼，在 `zrest_training` node 上右鍵 → **New Sub-Element**，Service Name 填 **rs06**
2. Handler List 掛 `ZCL_RS06_APP`
3. Activate Service
4. 用 `SPROX_HTTP_REQUEST` 測試（這題一定要用 `SPROX_HTTP_REQUEST` 或其他能自訂 **Request Body** 的工具，瀏覽器網址列沒辦法送 **POST body**）。URL 一樣要填完整網址（含 `http://` 與主機:port）：
`http://<主機>:<port>/sap/bc/zrest_training/rs06/flights?sap-client=130`，HTTP Method 選 **POST**，Req. Body 分頁貼 JSON：
 - 成功建立（**201**）：

```
``json {"carrid":"AA","connid":17,"fldate":"20260101","price":422.94,"currency":"USD"} ``
```

（**fldate** 這裡要用 **POST body** 傳入的原始格式，`/UI2/CL_JSON` 反序列化 **DATS** 欄位吃的是不帶分隔符的 8 碼數字，跟 **rs04** 路徑參數的輸入格式一致；已確認這組 `carrid/connid/fldate` 組合在資料庫裡還不存在）

- 缺必填欄位（**400**）：拿掉 `connid` 再送一次

```
``json {"carrid":"AA","fldate":"20260102","price":422.94,"currency":"USD"} ``
```

- `carrid` 不存在（**400**）：

```
``json {"carrid":"ZZ","connid":17,"fldate":"20260101","price":422.94,"currency":"USD"} ``
```

- 主鍵重複（**409**）：用一組資料庫裡已經存在的鍵值（例如 **rs05** 驗證過的 `AA/17/20190101`）

```
``json {"carrid":"AA","connid":17,"fldate":"20190101","price":422.94,"currency":"USD"} ``
```

1. 確認 **201** 回應的 **Resp. Header** 分頁裡有 **Location**：`/flights/AA/17/20260101`
2. 用 **rs04** 或 **rs05** 的 GET 端點驗證這筆新資料真的寫進 **SFLIGHT** 了（例如 **rs05** 的？
`carrid=AA&connid=17`，應該會看到多一筆 `2026-01-01`）

預期輸出（範例）

成功建立：HTTP 狀態碼 **201**，Response Header 含 **Location**：`/flights/AA/17/20260101`，Body：

```
{"carrid":"AA","connid":17,"fldate":"2026-01-01","price":422.94,"currency":"USD"}
```

缺必填欄位 / `carrid` 不存在：HTTP 狀態碼 **400**。

主鍵重複：HTTP 狀態碼 **409**。

團隊實務備註

- 這題會真的寫入 **SFLIGHT** 這張訓練用資料表，不像 rs01~rs05 全部都是唯讀查詢——測試完如果要清理，直接用 SE16/SE16N 手動刪掉 **AA/17/20260101** 這筆即可，**\$TMP** 套件的訓練環境沒有正式資料，不用擔心影響其他人
- **CORRESPONDING #(ls_flight)** 只會複製「兩邊都有、名稱相同」的欄位，**ls_flight** 只有 5 個欄位，轉進 **SFLIGHT** (十幾個欄位) 之後其餘欄位保持初始值 (**PLANETYPE** 空字串、**SEATSMAX** 等於 0) ——這在訓練情境沒問題，但如果是正式系統的資料表對必填欄位有業務邏輯要求 (例如「機型不能是空的」)，只做 DDIC 層級的必填檢查是不夠的，要在程式裡額外補業務規則驗證
- **400 vs 404 vs 409** 這三個狀態碼在這題 (連同 rs04/rs05) 分別對應到：**404** = 呼叫端要「讀」一個不存在的資源、**400** = 呼叫端「送來的資料」本身不合法 (缺欄位、引用了不存在的外鍵)、**409** = 呼叫端的請求沒錯，但跟系統目前狀態衝突 (想建立的東西已經存在) ——把這三者分清楚是設計一個像樣的 REST API 的基本功
- CSRF Token 的完整流程 (沒有關閉的情況下) 是：先發一個 GET 請求，Header 帶 **X-CSRF-Token: fetch**，伺服器回應的 Header 會帶回一個真正的 token；接下來的 POST/PUT/DELETE 請求的 Header 要帶 **X-CSRF-Token: <剛剛拿到的值>** 才會通過驗證——**SPROX_HTTP_REQUEST** 沒有自動處理這個兩階段流程的功能，這也是這題選擇關閉 CSRF 檢查的直接原因 (見上方說明)

思考題

1. 如果把 **carrid** 存在性檢查拿掉，只留必填欄位檢查，POST 一筆 **carrid** 亂打的資料 (例如 "XX") 會發生什麼事？**SFLIGHT** 有沒有 DDIC 層級的外鍵約束會擋下這筆資料？ (提示：回顧 [.claude/rules/sap-adt-mcp.md](#) 第 10 節提過的「DDIC 外鍵只在 Dynpro/SM30 畫面輸入層級生效，Open SQL 完全不受影響」)
2. 這題的 **Location** header 值是用字串組出來的相對路徑 (**/flights/AA/17/20260101**)，沒有帶 **http://<主機>:<port>** 前綴——如果呼叫端真的想直接把 **Location** 的值當網址呼叫下一個請求，這樣夠用嗎？正式的 REST API 通常會怎麼處理 **Location** header 的網址完整性？
3. 如果呼叫端在同一秒鐘內對同一組 **carrid/connid/fldate** 送出兩個 POST 請求 (例如網路重送)，有可能兩個請求都通過「主鍵不存在」的檢查、但只有一個 **INSERT** 會成功？這種「檢查」跟「寫入」之間的時間差 (**race condition**) 在資料庫層面要怎麼處理？ (提示：**INSERT** 語句本身在資料庫層級是有主鍵約束保護的，這題的程式邏輯是否也依賴了這個保護，還是只靠應用層的邏輯判斷？)

答案

見 **zcl_rs06_app.clas.abap**、**zcl_rs06_flights.clas.abap** (SAP 端物件 **ZCL_RS06_APP / ZCL_RS06_FLIGHTS**)。SICF Service 路徑 **/sap/bc/zrest_training/rs06**。